PathScope

A C++ study of how shortest-path algorithms behave on real road networks

DAA Lab Evaluation Project — Case Study

The name works because the tool lets you see how far each algorithm "scopes" the map before finding the answer. Alternatives: RouteLens, MargDarshak (Hindi for "path guide"), NavForge. Search GitHub before committing so the repo name does not clash with an existing one.

1. Problem

Navigation apps must find the shortest route between two places in a network of thousands of junctions and roads. Many algorithms solve this, but they differ a lot in speed, memory use and the kinds of graphs they handle. Choosing the wrong one wastes time on large maps, or gives wrong answers when edge weights are negative.

2. Objective

Build a C++ route planner that solves the same shortest-path problem with four algorithms and measures how they compare in running time, memory and nodes explored, as the map grows.

3. Approach

The road network is modelled as a weighted graph: junctions are vertices, roads are edges, and distance or travel time is the weight. The system implements:

- **Dijkstra (min-heap):** the standard choice for non-negative weights.
- **A*:** Dijkstra guided by a heuristic (straight-line distance) so it explores fewer nodes.
- **Bellman-Ford:** slower, but handles negative weights and detects negative cycles.
- **Floyd-Warshall:** computes the shortest paths between all pairs at once.

Input graphs come from a file or a random generator with adjustable size and density. A benchmark mode times each algorithm with `std::chrono`, counts nodes visited and edge relaxations, and writes the results to CSV for plotting.

4. Complexity Summary

Algorithm	Time	Space
Dijkstra	O((V+E) log V)	O(V)
A*	Heuristic-dependent; worst case like Dijkstra	O(V)
Bellman-Ford	O(V·E)	O(V)
Floyd-Warshall	$O(V^3)$	$O(V^2)$

5. Experiments

Run all four algorithms on graphs of increasing size (for example 100 to 10,000 nodes) and on both sparse and dense graphs. Record running time, nodes visited and relaxations, then plot input size against each metric and compare with the theoretical curves.

6. Results (fill in from your own runs)

Do not write numbers until you have measured them. The report should state:

- How much faster A* is than Dijkstra, and how many fewer nodes it visits (your measured %).
- The input size at which Floyd-Warshall becomes impractical.
- Whether Bellman-Ford's gap to Dijkstra grows on dense graphs.
- Whether the measured curves match the theoretical ones, and the reasons if they do not (cache effects, heap overhead, constant factors).

7. Key Learnings

- The best algorithm depends on the graph: sparse vs. dense, negative weights or not, one query vs. all pairs.
- Good heuristics can beat a better asymptotic bound in practice.
- Measured performance also depends on constants, memory layout and implementation, not only Big-O.

8. Future Scope

Bidirectional Dijkstra, traffic-aware dynamic weights, loading real OpenStreetMap data, and a simple visual front end.